

Reflection Agents With LangGraph | Agentic LLM Based Applications

9 min read · Apr 2, 2024

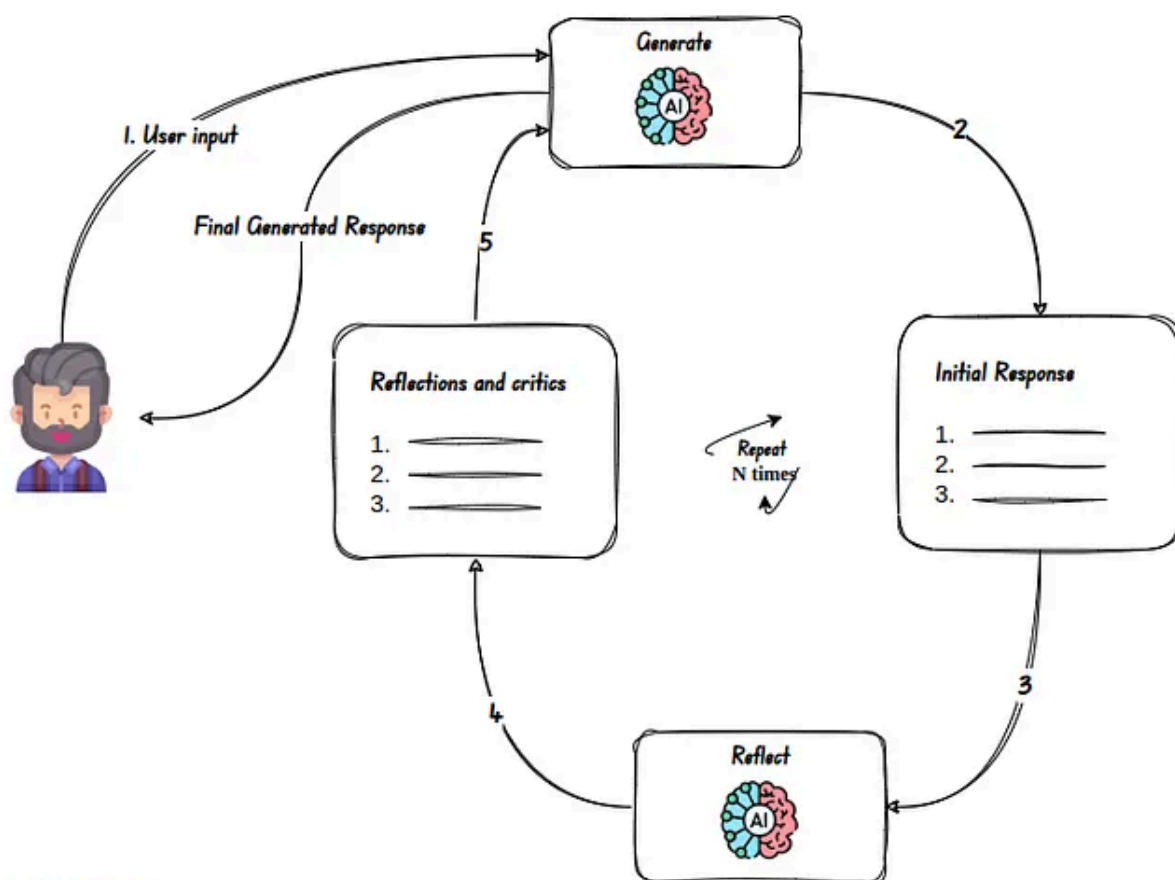


Prince Krampah

Follow

Listen

Share



Code With Prince

Open in app ↗

Sign up

Sign in

Medium

Search



Hello there, welcome back to another exciting series of article posts diving in the the fascinating world of agents. In this new series, we'll be delving into reflection based agents and understanding the power of reflection in LLM based systems. We'll not only go over the theoretical section, but also the implementation part of it. Let's take a look at what we'll be going over in this first article in the series of articles that I'll be publishing once a week on reflection agents in LLM based applications.

In this first article, we'll go over what is reflection in LLM based applications. Types of reflection agents and their implementations. When to use reflection agents and much more.

What Is Reflection?

Reflection is a prompting strategy used to improve the quality and success rate of agents and similar AI systems. It involves prompting an LLM to reflect on and critique its past actions, sometimes incorporating additional external information such as tools and observations.

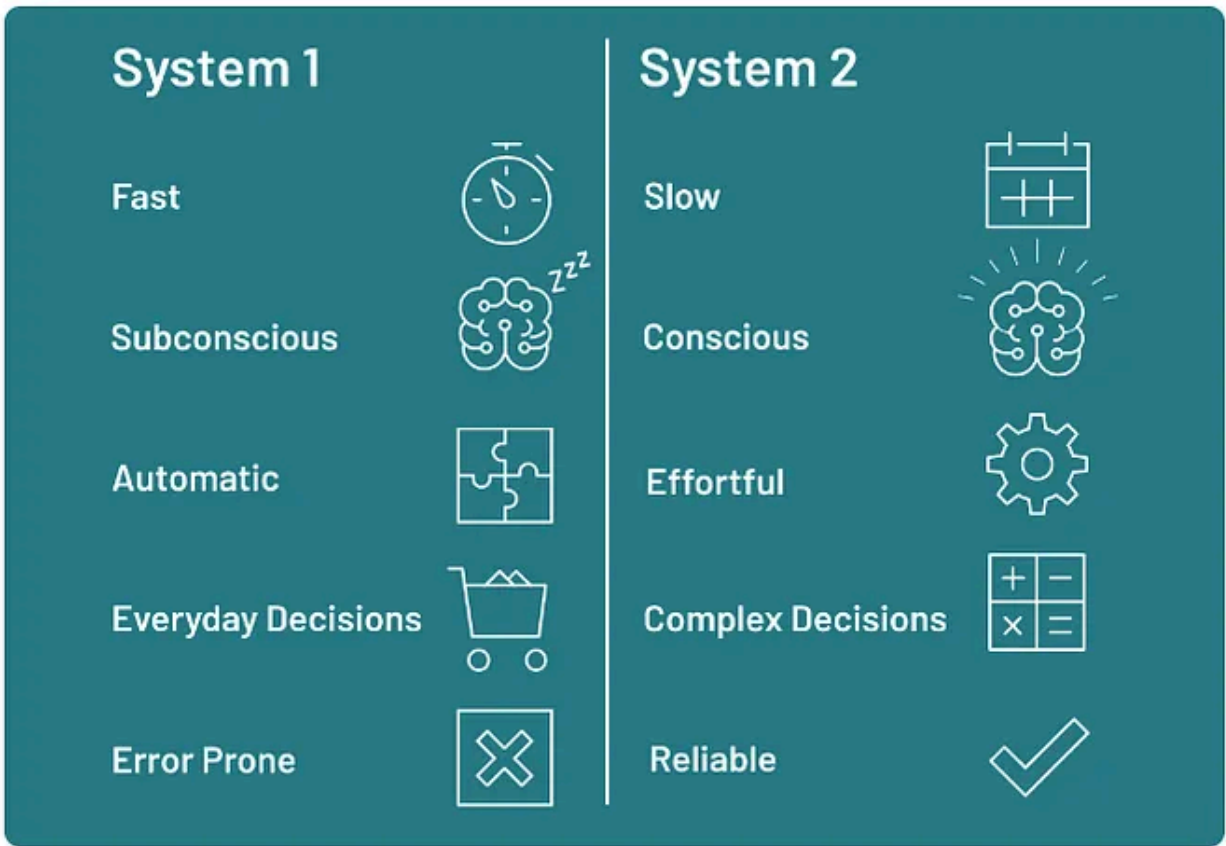
Imagine an LLM based application like ChatGPT. If you ask it to write you code. It immediately begins to spit out content of the code. Do you know any programmer that just writes code that fast and never tests it, but pushes it into production? Forget about other programmers, just maybe they are that intelligent. Can you personally do that? Of course not, highly unlikely.

What's why most of the time, code from tools like ChatGPT are prone to a tonne of errors. The code is sometimes never tested to ensure it runs well. I don't know what the back of ChatGPT is configured for, but I'm looking at the results from it. The code is so prone to error and never runs without an error. What does that communicate to you? The code is just generated and never tested before its output to you. Even considering the time it took to get the code generated shows it's just an AI thinking out of the blue and the code is never tested and run to ensure it actually works.

A better alternative to this is to get AI and LLM based systems to reflect on their own answers before outputting anything to the final user. Code should be tested and ran to ensure no errors before it's outputted to the user. The LLM based systems should be able to internally test their code to ensure it runs, fix all the bugs and go over it in an iterative manner to ensure, code is generated, tested, errors fixed, tested again, errors fixed... until the code runs well. Then and only then should the final output

be presented to the final user. That iterative approach of the LLM looking into what it generated and iterating over it to make it better is the whole concept of Reflection and reflection agents.

A simple way to understand this is the “system 1” and “system 2” architecture, credits to the LangChain blog. In such an architecture, system 1 does all the reactive and instinctive thinking while on the other end, system 2 does the reflection and reflexion part (What is reflexion? Don’t worry about that just yet). LLM for the longest time have been playing the system 1 side of things. Introducing reflection in prompting, we can usher LLM applications closer to system 2 like way of behaving.



[LangChain Blog](#)

So basically to achieve reflection in LLM based systems, we have multiple LLM agents. Some that generate content(Generative part of LLMs). Others that reflect on generated content and give constructive criticism that can then be used to optimize the next generation in the cycle to make the content better. Let’s take a look into the types of these reflection types we can work with. There may be many more approaches considering the type of publishing this article. Keep this in mind.

Types Of Reflection Agents

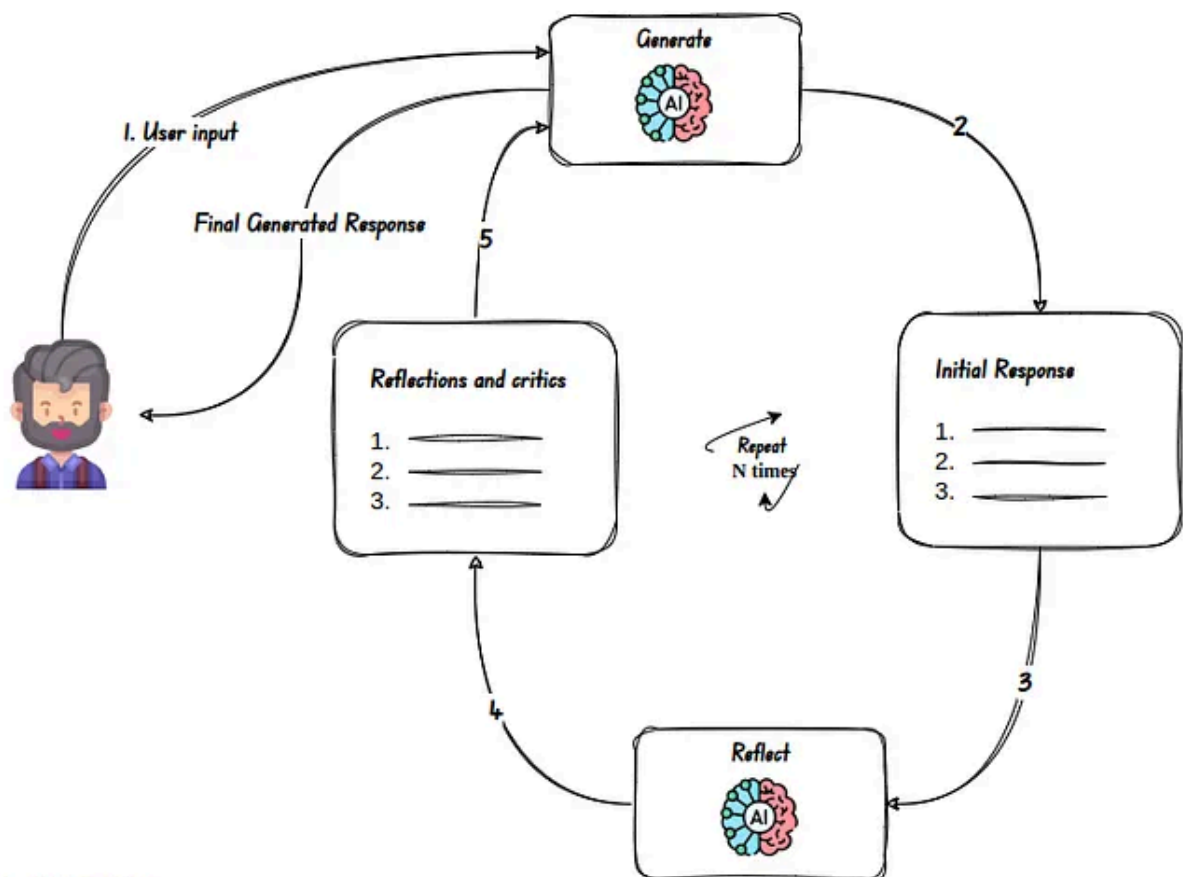
There are many types of reflection agents. In this article section, I'll only go over three main types of reflection agents. These include:

1. Basic Reflection Agents
2. Reflexion Agents
3. Language Agent Tree Search (LATS)

In this post, I am going to mainly focus on Basic Reflection Agents. In the upcoming articles, we'll take a look into the other two types. So what is a basic reflection agent?

Basic Reflection Agents

As the name suggests, this is the most basic form of reflection agents. In this setup we have two main LLM agents, one acting as the generator and the other acting as the reflector that criticizes the responses from the generator and suggests constructive feedback.

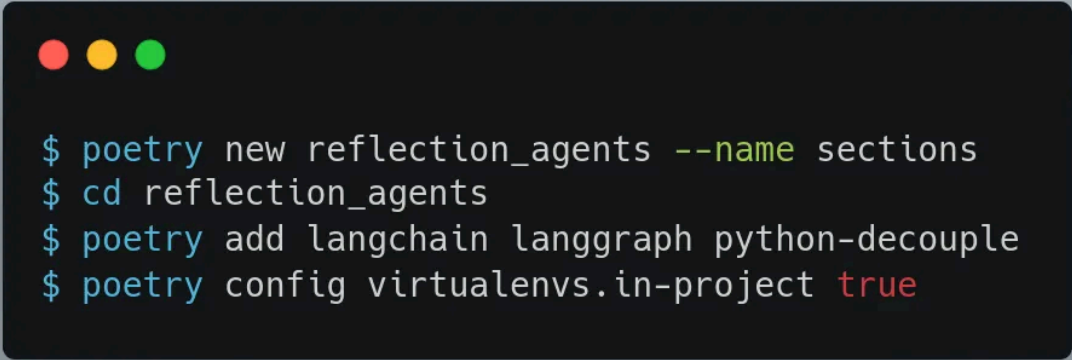


Code With Prince

Hope the image above gave you a glimpse into how reflection agents work from a basic reflection agents setup point of view. Now let's implement a basic reflection agent using LangGraph.

Setup And Installations

I'll be using poetry for the setup and installation, feel free to use whatever works with you. Here are the list of libraries we'll need to install to get running:



```
$ poetry new reflection_agents --name sections
$ cd reflection_agents
$ poetry add langchain langgraph python-decouple
$ poetry config virtualenvs.in-project true
```

Code With Prince

You'll also need to install `langchain-openai` using:

```
$ poetry add langchain-openai
```

Once you have this in place. Navigate into the `sections` folder and inside of that folder, add a new folder called `reflection_agent` inside of this folder, create a file called `basic_02.py`. I also have a `basic_01.py` file in there with some code that is explained in this video on my channel, but do not worry about that file for now. Also go ahead and create another file called `basic_03.py` file inside of the same `reflection_agent` directory, we'll use it in a bit.

Get Prince Krampah's stories in your inbox

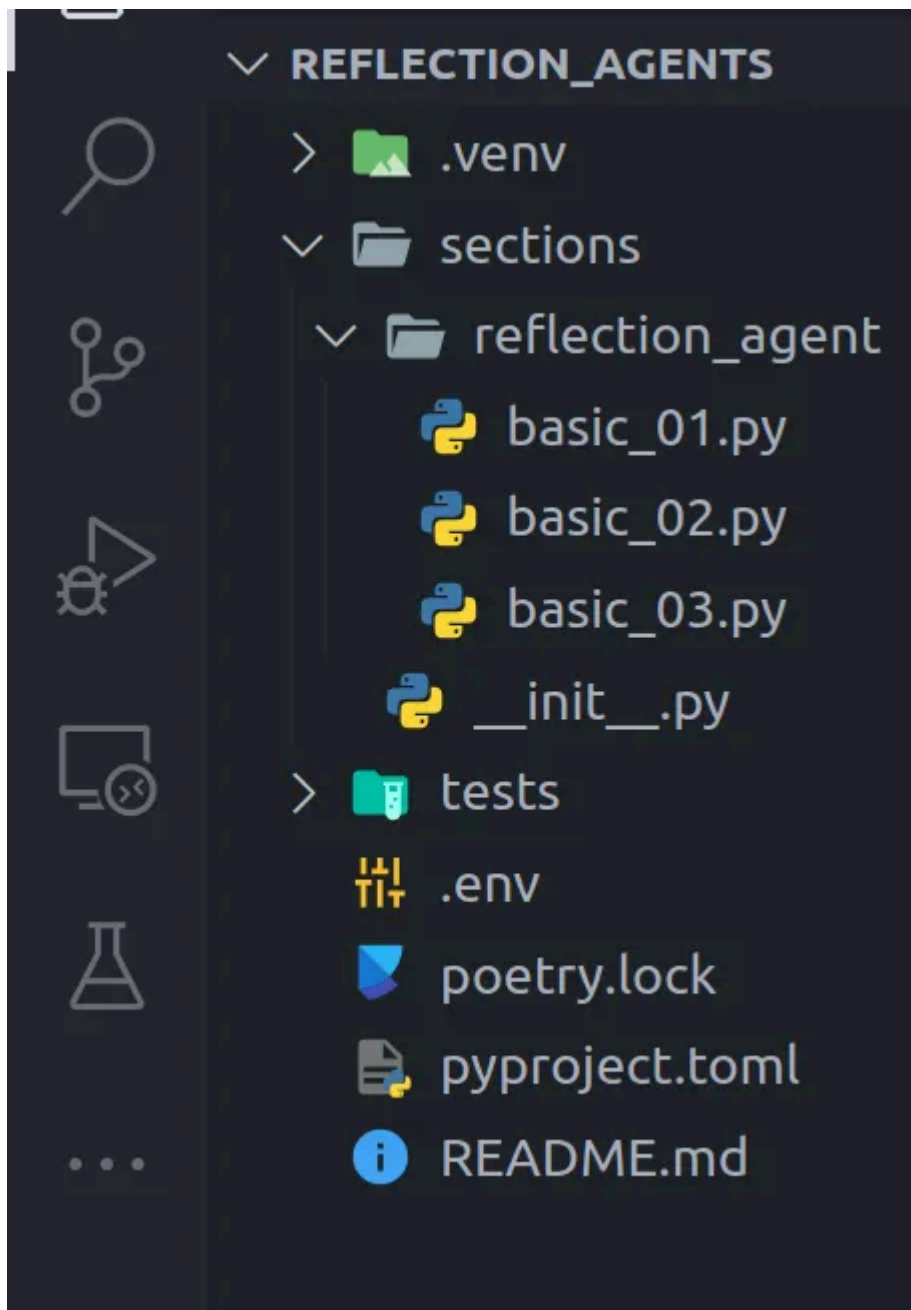
Join Medium for free to get updates from this writer.

Enter your email

Subscribe

☐ Remember me for faster sign in

In the root of the project folder, at the same level where we have the `sections` folder, add a file called `.env` we'll use this file to store our API keys.



Code With Prince

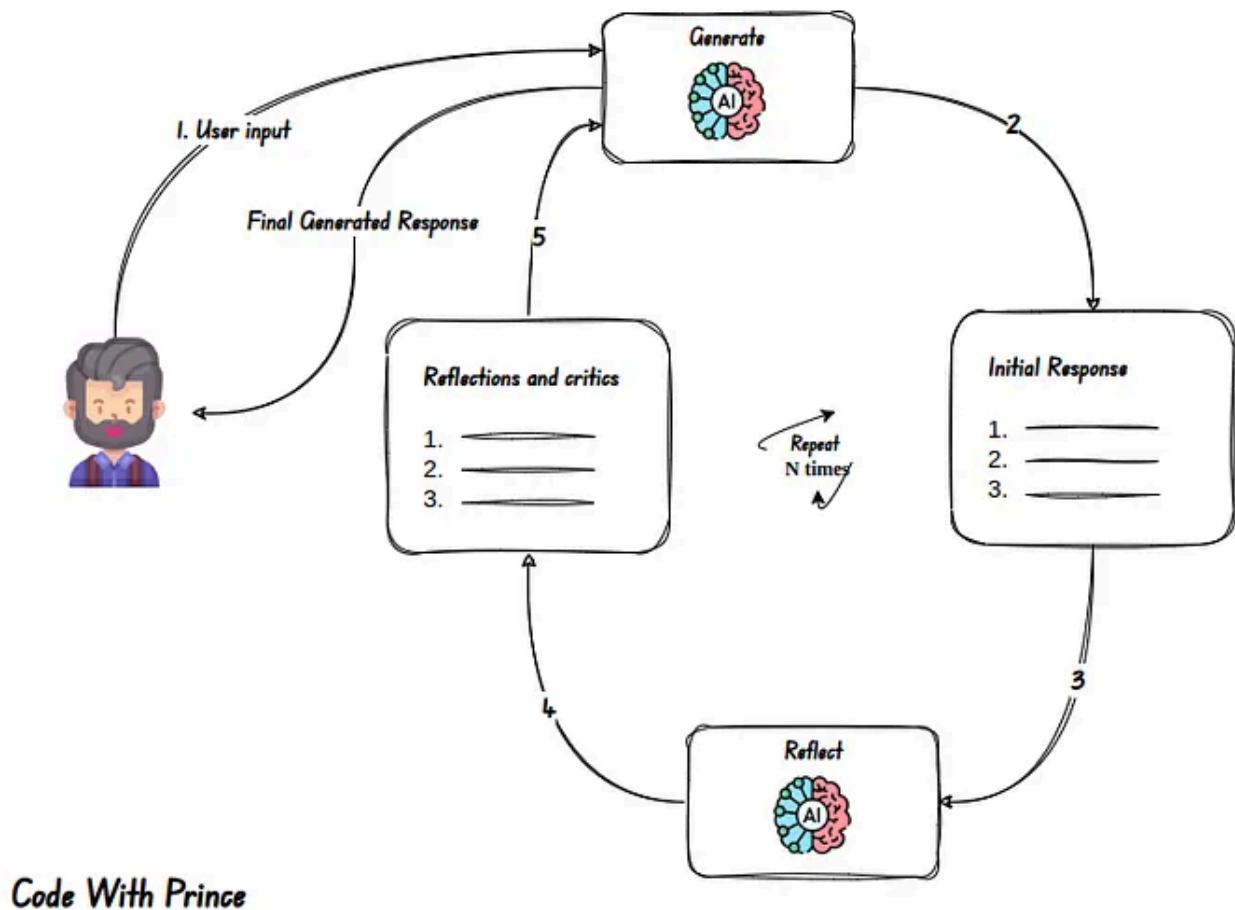
The content of your `.env` file should be as follows:

OPENAI_API_KEY=sk-3ilkqouroqujlfjalflajfldjl

Make sure you have your actual OpenAI API key in there. Once you have all that done with the setup. We are ready to rumble with coding! Let's get into it.

Creating Generate And Reflection Agents

Let's move ahead and implement the two main agents in the basic reflection architecture. Again I'll drop the image here below to give you an idea on what we are doing:



Code With Prince

The generator will do the generative part of an LLM, generating answers to user queries. Then the generated response (Initial Response) will be passed on to the reflect agent to reflect on it and give us constructive feedback. The constructive feedback will then be passed on to the generated agent to fine tune its initial response, we'll do this in a loop for N number of times.

So, let's move ahead and create these two agents and have the reflect agent critique the initial response manually, then we'll move on to doing the same with LangGraph in a more sophisticated and automated manner. Open the `basic_02.py` file and have the following code inside of it.

```
from langchain_core.messages import HumanMessage, SystemMessage
from decouple import config
from langchain_openai import ChatOpenAI

from typing import List

from langchain_core.prompts import ChatPromptTemplate, MessagesPlaceholder
```

Generate Agent

```
prompt = ChatPromptTemplate.from_messages(
    [
        # this needs to be a tuple so do not forget the , at the end and do not
        (
            "system",
            "You are an AI assistant researcher tasked with researching on a variable. "
            "Generate the best research possible as per user request."
            "If the user provides critique, respond with a revised version of the research."
        ),
        MessagesPlaceholder(variable_name="messages"),
    ]
)

llm = ChatOpenAI(api_key=config("OPENAI_API_KEY"))

generate = prompt | llm

research = ""
request = HumanMessage(
    content="Research on climate change"
)

# test the single generate agent
for chunk in generate.stream({"messages": [request]}):
    print(chunk.content, end="")
    research += chunk.content
```

Here we have the generating agent conduct research on climate change.

Reflect Agent

Once the initial response is generated, we'll have the Generate Agent critic it and give constructive feedback on how to make it better:

```
reflection_prompt = ChatPromptTemplate.from_messages([
    # this needs to be a tuple so do not forget the , at the end
    (
        "system",
        "You are a senior researcher"
        " Provide detailed recommendations, including requests for length,"
        " to an asistant researcher to help improve this researches",
    ),
    MessagesPlaceholder(variable_name="messages"),
])
reflect = reflection_prompt | llm

print("\n\n\n")
print("====Reflect====")
print("\n\n\n")

reflection = ""
for chunk in reflect.stream({"messages": [request, HumanMessage(content=research)]}):
    print(chunk.content, end="")
    reflection += chunk.content
```

This is only one round of the cycle. Where we have the initial responses being criticized by the reflect agent and we get back feedback. One thing you can notice is that we never passed the feedback back to the generated agent to improve on the initial response. You can do that in the code if you want to. You simply pass in the state of the conversation. A list of all the messages from the initial response to the critics from the reflect agent. Pass this state back to the generated agent. This can be a headache to handle manually. Good news: we have LangGraph that can take care of this for us. Let's see how LangGraph can come of use to us under this circumstance.

Basic Reflection Agent In LangGraph Implementation In Python

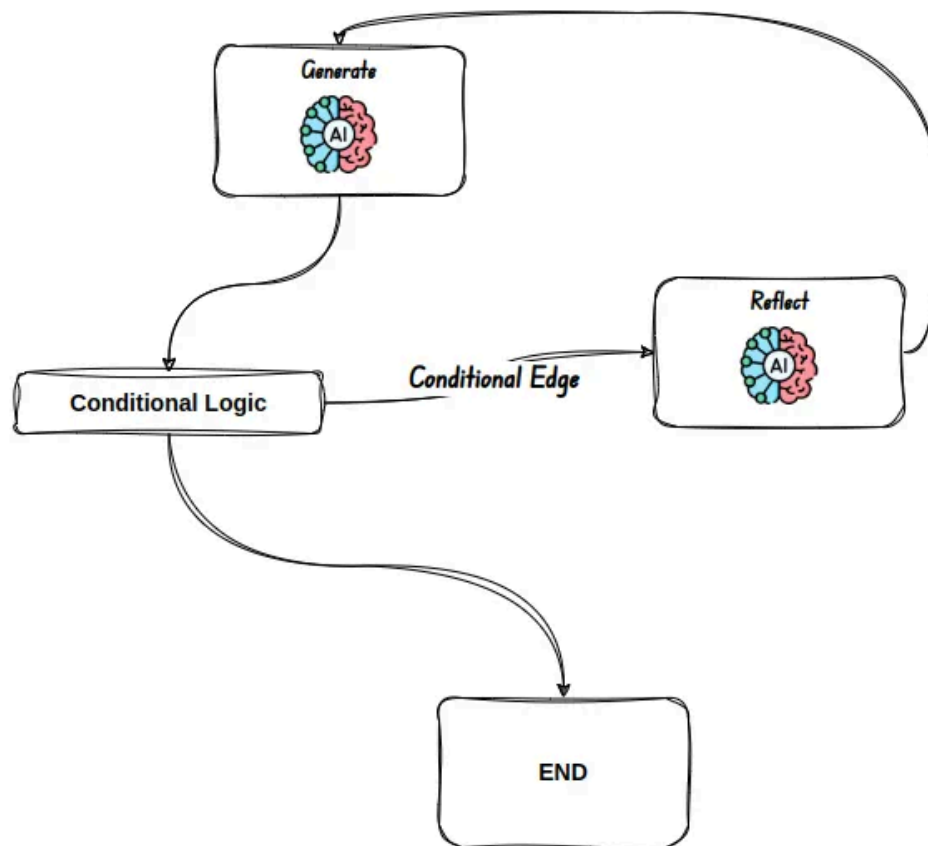
For this section, we'll have the code being written under the `basic_03.py` file.

Basically we'll have the same two agents. LangGraph just enables us to introduce

cycles in our code. Under which we can execute a task N number of times in a loop. The only difference we'll have in the code is something we call a conditional edge, what is a conditional edge?

Conditional Edge

A conditional edge is just an edge in our graph that we'll follow only under certain conditions. Example in our case, we want the execution of the cycle to go on for 6 rounds. Once this is reached a conditional edge we'll be executed and we'll exit the cycle and return the final response from the LLM.



Code With Prince

Once we have this out of the way, here's how we can implement this in code:

```
import asyncio
from langgraph.graph import END, MessageGraph
from typing import List, Sequence
from langgraph.graph import MessageGraph, END
from langchain_core.messages import BaseMessage, HumanMessage
from decouple import config
from langchain_openai import ChatOpenAI
```

```

from typing import List

from langchain_core.messages import AIMessage, BaseMessage, HumanMessage
from langchain_core.prompts import ChatPromptTemplate, MessagesPlaceholder

prompt = ChatPromptTemplate.from_messages(
    [
        (
            "system",
            "You are an AI assistant researcher tasked with researching on a va"
            " Generate the best research possible as per user request."
            " If the user provides critique, respond with a revised version of
        ),
        MessagesPlaceholder(variable_name="messages"),
    ]
)

llm = ChatOpenAI(api_key=config("OPENAI_API_KEY"))

generate = prompt | llm

reflection_prompt = ChatPromptTemplate.from_messages(
    [
        (
            "system",
            "You are a senior researcher"
            " Provide detailed recommendations, including requests for length,
            " to an asistant researcher to help improve this researches",
        ),
        MessagesPlaceholder(variable_name="messages"),
    ]
)

reflect = reflection_prompt | llm

async def generation_node(state: Sequence[BaseMessage]):
    return await generate.ainvoke({"messages": state})

async def reflection_node(messages: Sequence[BaseMessage]) -> List[BaseMessage]
    # Other messages we need to adjust
    cls_map = {"ai": HumanMessage, "human": AIMessage}

    # First message is the original user request. We hold it the same for all r
    translated = [messages[0]] + [
        cls_map[msg.type](content=msg.content) for msg in messages[1:]
    ]
    res = await reflect.ainvoke({"messages": translated})

    # this will be treated as a feedback for the generator

```

```

        return HumanMessage(content=res.content)

builder = MessageGraph()
builder.add_node("generate", generation_node)
builder.add_node("reflect", reflection_node)
builder.set_entry_point("generate")

def should_continue(state: List[BaseMessage]):
    if len(state) > 6:
        # End after 5 iterations
        return END
    return "reflect"

builder.add_conditional_edges("generate", should_continue)
builder.add_edge("reflect", "generate")
graph = builder.compile()

async def stream_responses():
    async for event in graph.astream(
        [
            HumanMessage(
                content="Research on climate change"
            )
        ],
    ):
        print(event)
        print("----")

asyncio.run(stream_responses())

```

Yup! that's all we needed to do to implement a basic reflection agent. You can find the video on this article on my YouTube channel, Code With Prince:

Conclusion

Congratulations for making it this far, I hope this has given you the understanding of function calling in Autogen. Hope you find use cases for it in your day to day use of multi-agent LLM based applications.

Other platforms you can reach out to me:

1. [YouTube](#)
2. [Twitter](#)
3. [LinkedIn](#)
4. [Discord](#)

Happy coding! and see you next time, the world keeps spinning.

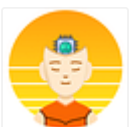
Reflection Agents

Agentic Ai

Agents

Langgraph

Reflexion



Follow

Published in AI monks.io

6.4K followers · Last published 2 hours ago

Almonks (<https://medium.com/aimonks>) is an AI-Educational Publication.



Follow

Written by Prince Krampah

1.3K followers · 25 following

Hello there , am Prince a full-stack web developer, data science enthusiast, lover of Python Programming with a deep interest in deep learning, computer vision
